CHAPTER 10

The Modern Data Warehouse

In Part II of this book, you learned about relational data warehouses (RDWs) and data lakes, two key components of the data management landscape. Now, let's consider the bustling world of modern business. Every day, organizations must sift through immense amounts of data to gain insights, make decisions, and drive growth. Imagine a city supermarket switching from traditional databases to a modern data warehouse (MDW). Managers can now access real-time inventory data, predict shopping trends, and streamline the shopping experience for their customers. That's the power of an MDW. It blends the best of both worlds: the structure of RDWs and the flexibility of data lakes.

Why should you care about MDWs? Because they are at the heart of our rapidly evolving data ecosystem, enabling organizations to harness the information they need to innovate and compete. In this chapter, I'll clarify what MDWs are and what you can achieve with them, and I'll show you some important considerations to keep in mind. We'll journey through the architecture, functionality, and common stepping stones to an MDW, concluding with an insightful case study. Let's dive into the world of modern data warehouses where data is more than just numbers—it's the fuel for success.

The MDW Architecture

Figure 10-1 illustrates the hybrid nature of the MDW, which combines an RDW with a data lake to create an environment that allows for flexible data manipulation and robust analytics.

As Chapter 4 detailed, an RDW operates on a top-down principle. Building one involves significant preparation in building the warehouse before you do any data loading (schema-on-write). This approach helps with historical reporting by enabling analysts to delve into descriptive analytics (unraveling *what* happened) and diagnostic analytics (probing *why* it happened).

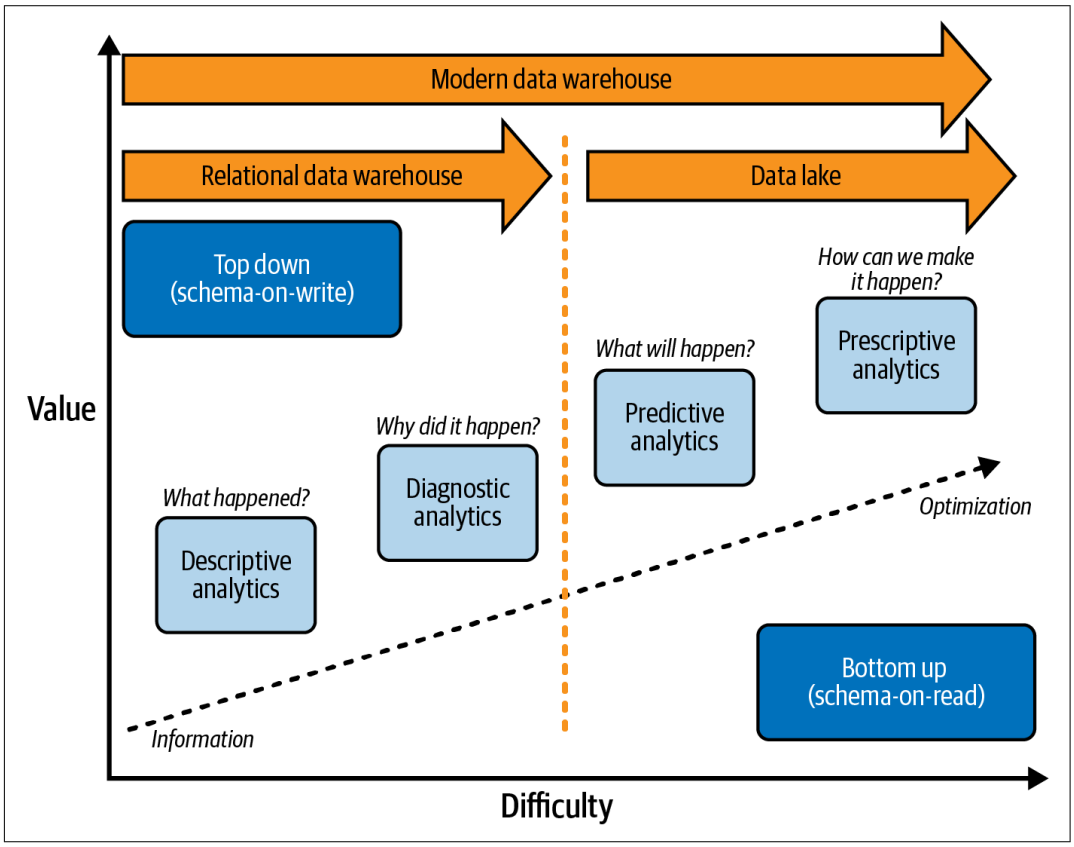

Figure 10-1. The full range of analytics facilitated by a modern data warehouse

In contrast, a data lake is defined by its bottom-up philosophy, as described in Chapter 5. Minimal up-front work is required to begin utilizing the data (schema-on-read), allowing rapid deployment of machine learning models to explore predictive analytics (forecasting what will happen) and prescriptive analytics (prescribing solutions to make desired outcomes occur).

Within an MDW architecture, a data lake is not merely a repository for storing vast amounts of information and training ML models; it's a dynamic component, responsible for data transformation and the other purposes explored in Chapter 5. It may act, for instance, as a receiving hub for streaming data, a gateway for users to do quick data exploration and reporting, and a centralized source to maintain a single version of truth.

An essential distinguishing feature of the MDW architecture is that *at least some* of its data must be replicated to an RDW. Without this duplication, it would be a data lakehouse architecture—a topic for Chapter 12.

In the mid-2010s, many organizations tried to use the data lake for all data use cases and bypass using an RDW altogether. After that approach failed, the MDW became the most popular architecture, driven by an exponential increase in data volume, variety, and velocity. Organizations in many sectors, including healthcare, finance, retail, and technology, recognized the need for a more agile and scalable approach to data storage and analysis. Their RDWs were no longer sufficient to handle the challenges of big data, but MDWs offered a flexible integration of RDWs and data lakes. Key players emerged as leading providers—Microsoft with Azure Synapse Analytics, Amazon with Redshift, Google with BigQuery, and Snowflake—revolutionizing the way businesses access and utilize their data.

As streaming data, large data, and semi-structured data became even more popular, large enterprises started to use the data fabric and data lakehouse architectures discussed in Chapters 11 and 12. However, for customers with not much data (say, under 10 terabytes), the MDW is still popular and will likely stay that way, at least until data lakes can operate as well as RDWs can.

In fact, there are still companies building cloud solutions that aren't using data lakes at all, mostly for use cases where the company has small amounts of data and is migrating from an on-prem solution that did not have a data lake. I still see this as an acceptable approach, with the caveat that the company has to be *absolutely sure* that the data warehouse will not grow much (which is very hard to say for certain). Also, if you have such a small amount of data, it may be feasible to go with a cheaper symmetric multiprocessing (SMP) solution for your RDW than a massively parallel processing (MPP) solution (see Chapter 7).

Now let's dig into the details of the MDW. Figure 10-2 shows a typical flow of data through the data lake and RDW in an MDW.

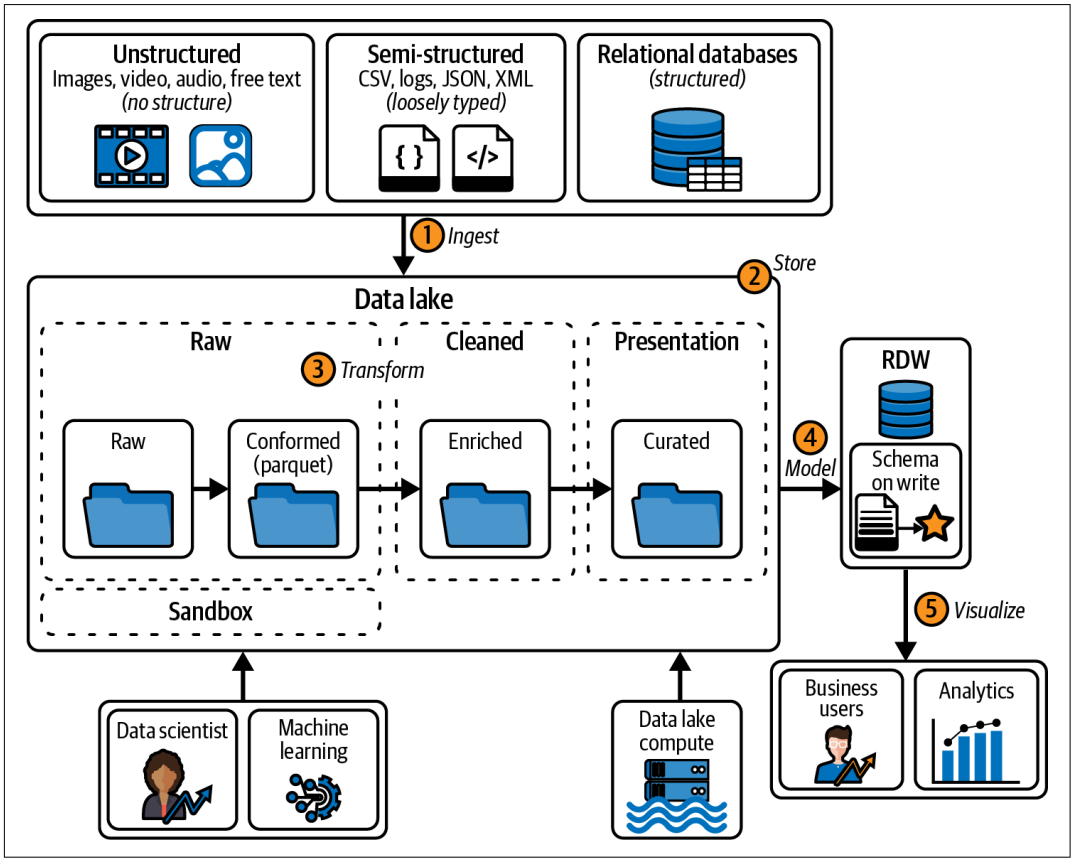

Figure 10-2. Overview of the journey that data takes through a modern data warehouse architecture

As data travels through the MDW architecture, it goes through five stages, which are numbered in Figure 10-2. Let's follow the data through each stage to give you a tour of the MDW architecture:

Step 1: Ingestion
: An MDW can handle just about any type of data, and it can come from many sources, both on-prem and in the cloud. The data may vary in size, speed, and type; it can be unstructured, semi-structured, or relational; it can come in batches or via real-time streaming; it can come in small files or massive ones. This variety can be a challenge during data ingestion.

Step 2: Storage
: Once the data is ingested, it lands in a data lake that contains all the various layers explained in Chapter 5, allowing end users to access it no matter where they are (provided they have access to the cloud). Every cloud provider offers unlimited data lake storage, and the cost is relatively very cheap. Baked into the data

lake are high availability and robust disaster recovery, along with many options for security and encryption.

Step 3: Transformation
: The essence of a data lake is that it's a storage hub. However, to truly make sense of and work with the data, you need some muscle—which is where computing power comes in. A significant advantage of the MDW is its clear distinction between storing data (in the data lake) and processing it (using computing power). Keeping the two separate frees you to pick and choose from a variety of computing tools, from cloud providers or other sources. These tools, designed to fetch and understand data from the lake, can read data in various formats and then organize it into a more standardized one, like Parquet. This separation offers flexibility and enhances efficiency.

  The computing tool then takes the files from the conformed layer, transforms the data (enriching and cleaning it), and stores it in the cleaned layer of the data lake. Finally, the computing tool takes the files from the enriched layer and performs more transformations for performance or ease of use (such as joining data from multiple files and aggregating it) and then writes it to the presentation layer in the data lake.

Step 4: Data modeling
: Reporting directly from the data in a data lake can be slow, unsecure, and confusing for end users. Instead, you can copy all or some of the data from the data lake into a relational data warehouse. This means you will create a relational model for the data in the RDW, where the model is usually in third normal form (see Chapter 8). You may also want to copy it into a star schema within the RDW for performance reasons and simplification.

Step 5: Visualization
: Once the data is in the RDW in an easy-to-understand format, business users can analyze it using familiar tools, such as reports and dashboards.

For the ingestion stage, you will need to determine how often to extract data and whether to use an incremental or a full extract (see Chapter 4). Keep in mind the size and bandwidth of the pipeline from the data sources to the cloud where the data lake resides. If you need to transfer large files and your bandwidth is small, you might need to upload data multiple times a day in smaller chunks instead of in one big chunk once a day. Your cloud provider might even have options for purchasing more bandwidth to the internet.

At various steps along this journey, data scientists can use machine learning to train and build models from the data in the MDW. Depending on their needs, they can take data from the raw, cleaned, or presentation layers in the data lake, from a

sandbox layer in the data lake (a dedicated space to do data experimentation, exploration, and preliminary analysis without affecting the other layers), or from the RDW.

There are some exceptions to the journey I've just laid out in which data flows through the MDW architecture, depending on the characteristics of the source data. For example, not all data in the presentation layer of the data lake needs to be copied to the RDW, especially with newer tools making it easier to query and report off data in a data lake. This means that in the visualization stage, the business user doesn't *have* to go to the RDW for all the data—they can access the data in the data lake that has not been copied to the RDW.

Another exception is that not all source data has to be copied to the data lake. In some cases, particularly new projects being built in the cloud, it's faster to copy data from the source right to the RDW, bypassing the data lake, especially for structured (relational) source data. For instance, reference tables that are used as dimension tables don't need to be cleaned, are full extracts, and can easily be retrieved from the source if an ETL package needed to be rerun. After all, it could be a lot of work to extract data from a relational database, copy it to the data lake (losing valuable metadata for that data, such as data types, constraints, and foreign keys), only to then import it into another relational database (the data warehouse).

This is particularly true of a typical migration from an on-prem solution that is not using a data lake and has many ETL packages that copy data from a relational database to an RDW. To reduce the migration effort, you'd need to modify the existing ETL packages only slightly—changing just the destination source. Over time, you could modify all the ETL packages to use the data lake. To get up and running quickly, however, you might only modify a few slow-running ETL packages to use the data lake and do the rest later.

Source data that bypasses the data lake would miss out on some of its benefits—in particular, being backed up, in case you need to rerun the ETL package. Bypassing can also place undue strain on the RDW, which then becomes responsible for data cleansing. Additionally, the people using the data lake will find that data missing, preventing the data lake from serving as the single source of truth.

Pros and Cons of the MDW Architecture

MDWs offer numerous benefits and some challenges. The pros include:

Integration of multiple data sources
: MDWs can handle both structured and unstructured data from various sources, including RDWs and data lakes, offering a comprehensive view of the information.

Scalability
: MDWs are designed to grow with the business, easily scaling to handle increased data loads.

Real-time data analysis
: MDWs facilitate real-time analytics, allowing businesses to make timely decisions based on current data.

Improved performance
: By leveraging advanced technology and optimized data processing, MDWs can offer faster query performance and insights retrieval.

Flexibility
: MDWs provide flexibility in data modeling, allowing for both traditional structured queries and big data processing techniques.

Enhanced security
: The leading MDW providers implement strong security measures to protect sensitive data.

Some of the cons of the MDW architecture are:

Complexity
: Implementing and managing an MDW can be complex, especially in hybrid architectures that combine different types of data storage and processing.

Cost
: The initial setup, ongoing maintenance, and scaling can be expensive, particularly for small- to medium-sized businesses. Plus, there will be extra costs for storage and for additional data pipelines to create multiple copies of the data (for both the RDW and data lake).

Skill requirements
: Utilizing an MDW to its full potential requires specialized knowledge and skills, potentially leading to hiring challenges or additional training costs.

Potential data silos
: Without proper integration and governance, MDWs can lead to *data silos*, or places where information becomes isolated and difficult to access across the organization.

Compliance challenges
: Meeting regulatory compliance in an environment that handles diverse data types and sources can be a challenging aspect of managing an MDW.

Vendor dependency
: If you're using a cloud-based MDW service, it may mean a dependency on a particular vendor, which could lead to potential lock-in and limit flexibility in the future.

As the data is copied within an MDW and moves through the data lake and into the RDW, it changes format and becomes easier and easier to use. This helps provide user-friendly self-service BI, where end users can build a report simply by dragging fields from a list to a workspace without joining any tables. The IT department must do some extra up-front work to make the data really easy to use, but that's often worth it, since it releases that department from being involved in (and likely a bottleneck for) all report and dashboard creation.

Combining the RDW and Data Lake

In an MDW, the data lake is used for *staging and preparing* data, while the RDW is for *serving*, *security*, and *compliance*. Let's look more closely at where the functionality resides when you're using both an RDW and data lake.

Data Lake

In a data lake, data scientists and power users—specifically those with higher technical skills—will have exclusive access, due to the complex folder-file structure and separation of metadata. Data lakes can be difficult to navigate, and access may require more sophisticated tools. The data lake's functions extend to batch processing, where data is transformed in batches, and real-time processing, which serves as the landing spot for streaming data (see Chapter 9). The data lake is also used to refine and clean data, providing a platform to use as much compute power as needed, with multiple compute options to select from. It also accommodates ELT workloads.

The data lake, in this framework, serves as a place to store older or backup data, instead of keeping it in the RDW. It's also a place to back up data from the data warehouse itself. Users can easily create copies of data in the data lake for sandbox purposes, allowing others to use and "play" in the data. The data lake also provides an opportunity to view and explore data if you're not sure what questions to ask of it; you can gauge its value before copying it to the data warehouse. It facilitates quick reporting and access to data, especially since it's schema-on-read so data can land swiftly.

Relational Data Warehouse

Business people often turn to RDWs as the place for non-technical individuals to access data, especially if they are accustomed to relational databases. These databases offer low latency, enabling much faster querying, especially if you're using MPP

technology (see Chapter 7). They can handle a large number of table joins and complex queries, and they're great for running interactive ad hoc queries because of the fast performance MPP technology provides. This allows you to fine-tune queries continuously, without long waits. RDWs can also support many concurrent users running queries and reports simultaneously.

The additional security features of RDWs in the MDW context include options such as row-level and column-level security. RDWs also provide plenty of support for tools—and since they have been around for much longer than data lakes, many more tools are available. Data lakes don't have the performance to support dashboards, but you can run dashboards against RDWs, thanks to their superior performance down to the millisecond. A forced metadata layer above the data requires more up-front work but makes self-service BI much easier. Generally, when you're building out an RDW, you already know what questions you want to ask of the data and can plan for them, making the RDW a powerful and versatile tool.

Stepping Stones to the MDW

Building an MDW is a critical but long and arduous endeavor that demands considerable investment in technology, human resources, and time. It represents the evolution of data management, where integration, accessibility, security, and scalability are paramount. While that process is starting, most organizations require interim solutions to address their current data warehousing needs. These solutions, which function as stepping stones to the fully operational MDW, ensure that the organization can still derive value from its data in the meantime and remain agile and responsive to business needs. They are not merely temporary fixes but vital components in a strategic migration.

Three common types of stepping-stone architectures are:

- EDW augmentation
- Temporary data lake plus EDW
- All-in-one

Each of these approaches offers unique benefits and challenges. Their suitability as pathways to an MDW can vary depending on the organization's needs, existing infrastructure, budget, and strategic objectives. Let's look at each one more closely.

EDW Augmentation

This architecture is usually seen when a company that has had a large on-prem EDW for a long time wants to get value out of its data, but its EDW can't handle "big data" due to a lack of storage space, compute power, available time in the maintenance window for data loading, or general support for semi-structured data.

How it works

You create a data lake in the cloud and copy the big data to it. Users can query and build reports from the data lake, but the primary data remains stored in the EDW, as shown in Figure 10-3.

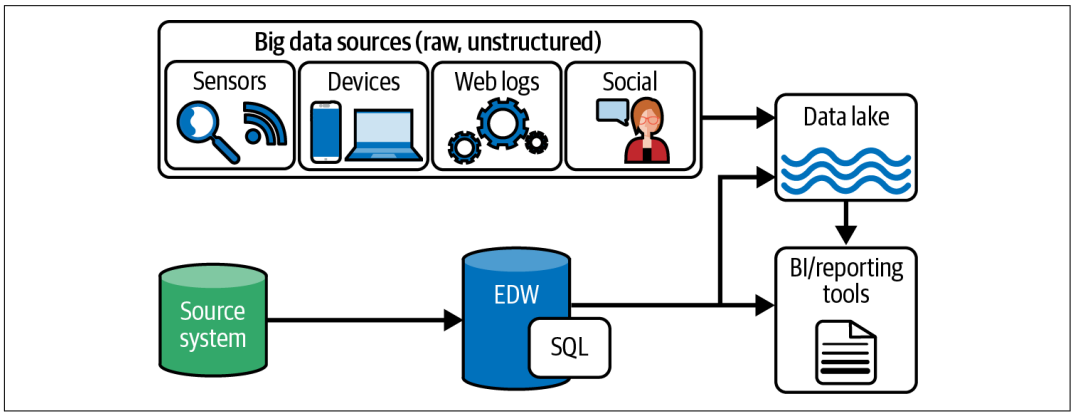

Figure 10-3. EDW augmentation architecture

Benefits

EDW augmentation architecture enhances data capacity and flexibility by leveraging a cloud data lake alongside an existing EDW. This strategy offers scalability and cost efficiency, creating opportunities for innovative analytics while maintaining current structures. It provides a balanced approach that aligns with business growth and continuity.

Challenges

This architecture does present some potential difficulties:

- If you need to combine data from the EDW with data in the data lake, you have to copy the data from the EDW to the data lake.
- Your existing query tools may not work against a data lake.
- You'll need a new form of compute to clean the data in the data lake, which may be expensive and require new skill sets.
- This architecture does not help you offload any of the EDW's workload.

Migration

This architecture can be the start of a phased approach to migrating the on-prem EDW to the cloud. After the architecture is up and running with the big data, you can begin migrating the data in the on-prem EDW to the data lake. Data from source

systems goes first to the data lake and then, if needed, to a new RDW in the cloud, which is part of a true MDW.

Temporary Data Lake Plus EDW

Companies usually use a temporary data lake with an EDW when they have an EDW and need to incorporate big data, but transforming it would take too much time. These companies want to reduce the EDW maintenance window by offloading data transformations.

How it works

This architecture uses a data lake, but solely as a temporary staging and refining area. It is not used for querying or reporting; that's all done from the EDW, as Figure 10-4 shows. This limited scope means the data lake can be incorporated much faster. Sometimes you can even copy EDW data to the data lake, refine it, and move it back to the EDW.

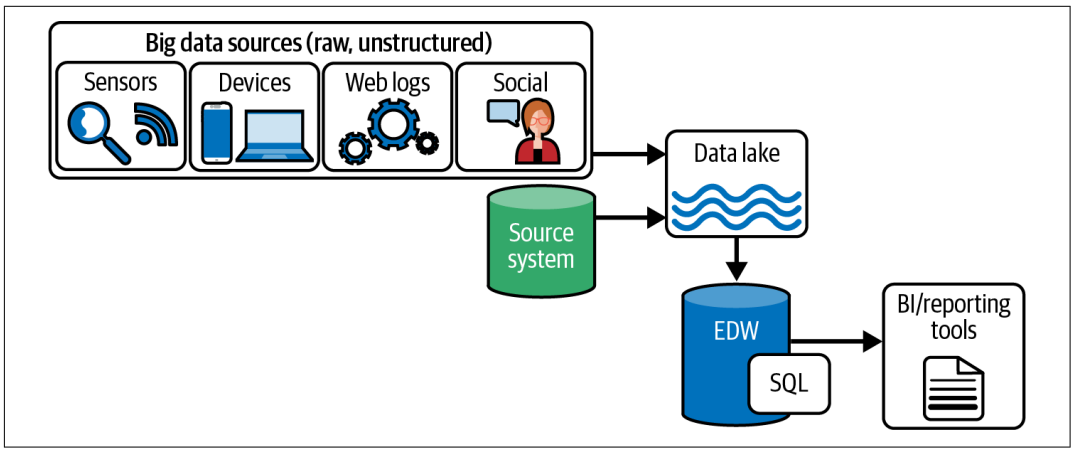

Figure 10-4. Using an EDW with a temporary data lake

Benefits

This architecture enables offloading data processing to the data lake, alleviating strain on the EDW and enhancing overall performance. Using other types of compute on the data lake can provide more speed and functionality, allowing for flexible handling of big data. This strategy offers a cost-effective solution for incorporating large datasets without disrupting existing EDW operations, making it an agile and scalable approach.

Challenges

Although you're using a data lake, this architecture does not give you the full benefits of a data lake (discussed in Chapter 5).

Migration

With just a few modifications, this architecture can evolve into a full-blown MDW, so it's a good stepping stone.

All-in-One

All-in-one architecture is typically adopted by organizations seeking a streamlined approach to data handling, such as startups or small businesses. It may be favored to do quick prototyping or to achieve specific short-term goals. It is also a good option when the primary users are technical experts who prefer an integrated platform.

How it works

All reporting and querying are done using the data lake, as Figure 10-5 shows. No RDW is involved. (All-in-one is closely tied to the data lakehouse architecture discussed in Chapter 12.)

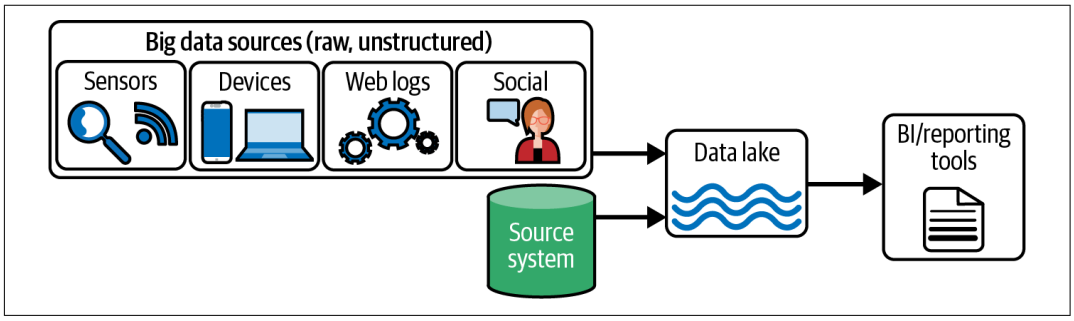

Figure 10-5. All-in-one architecture

Benefits

This approach allows for quick implementation and immediate results, benefits that are particularly attractive for technical teams aiming for rapid progress. By consolidating reporting and querying within the data lake, it simplifies the architecture, potentially reducing maintenance and integration complexities. This approach can be more agile and adaptable, accommodating various data types and fostering a more streamlined development process.

Challenges

Having no RDW involves trade-offs with respect to performance, security, referential integrity, or user friendliness.

Migration

For some companies and use cases, such as when the data in the data lake will be used only by data scientists, the data-lake-only approach may be fine. But to make it a stepping stone to an MDW, you'll need to add an RDW.

Case Study: Wilson & Gunkerk's Strategic Shift to an MDW

Wilson & Gunkerk, a fictitious midsized pharmaceutical company located in the US Midwest, had been relying on an on-prem solution to handle its data management requirements. With data volumes consistently under 1TB, the company never felt the need to transition to more advanced solutions like a data lake or an MDW.

Challenge

The company's need for actionable insights into market trends and customer behaviors became more pressing as its business environment became more competitive. The existing data system started showing its limitations, and the organization faced a dilemma: whether to upgrade to a more comprehensive data lake solution or an MDW. Another critical consideration was uncertainty about future data growth, particularly with new research and development projects in the pipeline.

Solution

After a thorough analysis, Wilson & Gunkerk decided to implement an MDW solution in the cloud. A few factors that led to this decision were:

- The existing data volume was well under 1TB, and projected growth was within 10TB.
- The migration to a cloud-based MDW offered a smoother transition from the existing on-prem solution without the need to adapt to an entirely new technology like a data lake.
- An MDW would be cost-effective. Wilson & Gunkerk opted for an SMP solution rather than a more expensive MPP option, since its data needs were modest and manageable.
- The chosen MDW provided robust performance and sufficient scalability to handle anticipated data growth within acceptable limits.

Outcome

The transition to the MDW was completed within a few months with minimal disruption to ongoing business processes. Wilson & Gunkerk realized immediate benefits:

Enhanced analytics
: The new system facilitated better data analytics, enabling more informed decision making. The company moved beyond simple historical analysis into predictive analytics. Before the MDW, insights were limited to past trends. The new system facilitated the integration of diverse data sources and the use of sophisticated data-mining algorithms. This allowed the company to create predictive models, such as forecasting drug efficacy based on patient demographics and genetic factors. The transition led to more nuanced decision making, offering a more precise approach to drug development and personalized medical solutions.

Cost savings
: The SMP solution provided excellent performance at a reduced cost compared to potential MPP solutions or a full-fledged data lake.

Future-proofing
: While keeping the door open to future transitions, the MDW allowed for gradual growth and adaptability without overinvestment in technology the company didn't immediately need.

Wilson & Gunkerk's case illustrates a balanced approach to data management solutions. For companies dealing with modest data volumes and seeking a scalable and cost-effective solution, MDWs can still present an appealing option. Each organization's needs and growth trajectory are unique, so a careful analysis of present and future requirements is vital to select the most appropriate data architecture.

Summary

This chapter started with a detailed description of the MDW and then described the five key stages of data's journey through an MDW: ingestion, storage, transformation, modeling, and visualization. These stages represent the lifecycle of data as it is initially captured, securely stored, refined for usability, modeled for insights, and finally visualized for easy interpretation and decision making.

I detailed the benefits of MDW, including aspects such as integration, scalability, real-time analysis, and security, while considering challenges like complexity, cost, and vendor dependency. We then delved into the combination of traditional RDWs with data lakes, highlighting the unique capabilities this blend provides and the enhanced flexibility it offers in dealing with various data types and structures.

I then provided a detailed exploration into three transitional architectures that companies often deploy as temporary solutions for their data-warehousing needs: enterprise data warehouse augmentation, temporary data lake plus EDW, and all-in-one. Each model serves as an interim measure while organizations work toward implementation of a full-fledged MDW.

I concluded with a case study of Wilson & Gunkerk, a midsized pharmaceutical company, and its strategic shift to an MDW.

The coming chapters will delve deeper into the data fabric and data lakehouse architectures, unpacking their benefits and potential use cases in the evolving landscape of data management.

CHAPTER 11
Data Fabric

The *data fabric* architecture is an evolution of the modern data warehouse (MDW) architecture: an advanced layer built onto the MDW to enhance data accessibility, security, discoverability, and availability. Picture the data fabric weaving its way throughout your entire company, accumulating all of the data and providing it to everyone who needs it, within your company or even outside it. It's an architecture that can consume data no matter the size, speed, or type. The most important aspect of the data fabric philosophy is that a data fabric solution can consume any and all data within the organization.

That is *my* definition of a data fabric; others in the industry define it differently. Some even use the term interchangeably with *modern data warehouse*! For instance, the consulting firm Gartner starts with a similar definition, writing that the data fabric is

> a design concept that serves as an integrated layer (fabric) of data and connecting processes. A data fabric utilizes continuous analytics over existing, discoverable and inferenced metadata assets to support the design, deployment and utilization of integrated and reusable data across all environments, including hybrid and multi-cloud platforms.

Where Gartner's view diverges from mine is in the view that data virtualization (which you learned about in Chapter 6) is a major piece of the data fabric technology that reduces the need to move or copy data from siloed systems. Gartner envisions an "intelligent data fabric" that leverages knowledge graphs and AI and ML for automating data discovery and cataloging, finding data relationships, orchestrating and integrating data, and auto-discovering metadata. However, much of the needed technology does not yet exist, and so far, neither does a data fabric that would satisfy Gartner's definition. I'm hopeful that technology will be available one day, but for now I recommend focusing on data fabric architectures as they exist today.

Data fabric is the most widely used architecture for new solutions being built in the cloud, especially for large amounts of data (over 10 terabytes). A lot of companies are "upgrading" their MDW architecture into a data fabric by adding additional technology, especially real-time processing, data access policies, and a metadata catalog. This chapter outlines eight technology features that take an MDW into data fabric territory. There's some ambiguity here; if you add three of those features, do you have an MDW or a data fabric? There is no bright line between the two. Personally, I call it a data fabric architecture if it uses at least three of those eight technologies. Many people use none of them but nonetheless call their solution a data fabric because they abide by the data fabric philosophy (I think this is perfectly fine).

This chapter gives you an overview of the data fabric architecture and its eight key components: data access policies, a metadata catalog, master data management (MDM), data virtualization, real-time processing, APIs, services, and products.

The Data Fabric Architecture

The data fabric architecture contains all of the components of the MDW but adds several features. Figure 11-1 shows a diagram of this architecture, with the stages of the data's journey labeled by number like they were in Figure 10-2. Data in a data fabric follows the same process as in an MDW; as you may recall from Chapter 10, those stages are (1) ingestion, (2) storage, (3) transformation, (4) modeling, and (5) visualization.

So what does a data fabric offer above and beyond the MDW? This section will take a look at its added features.

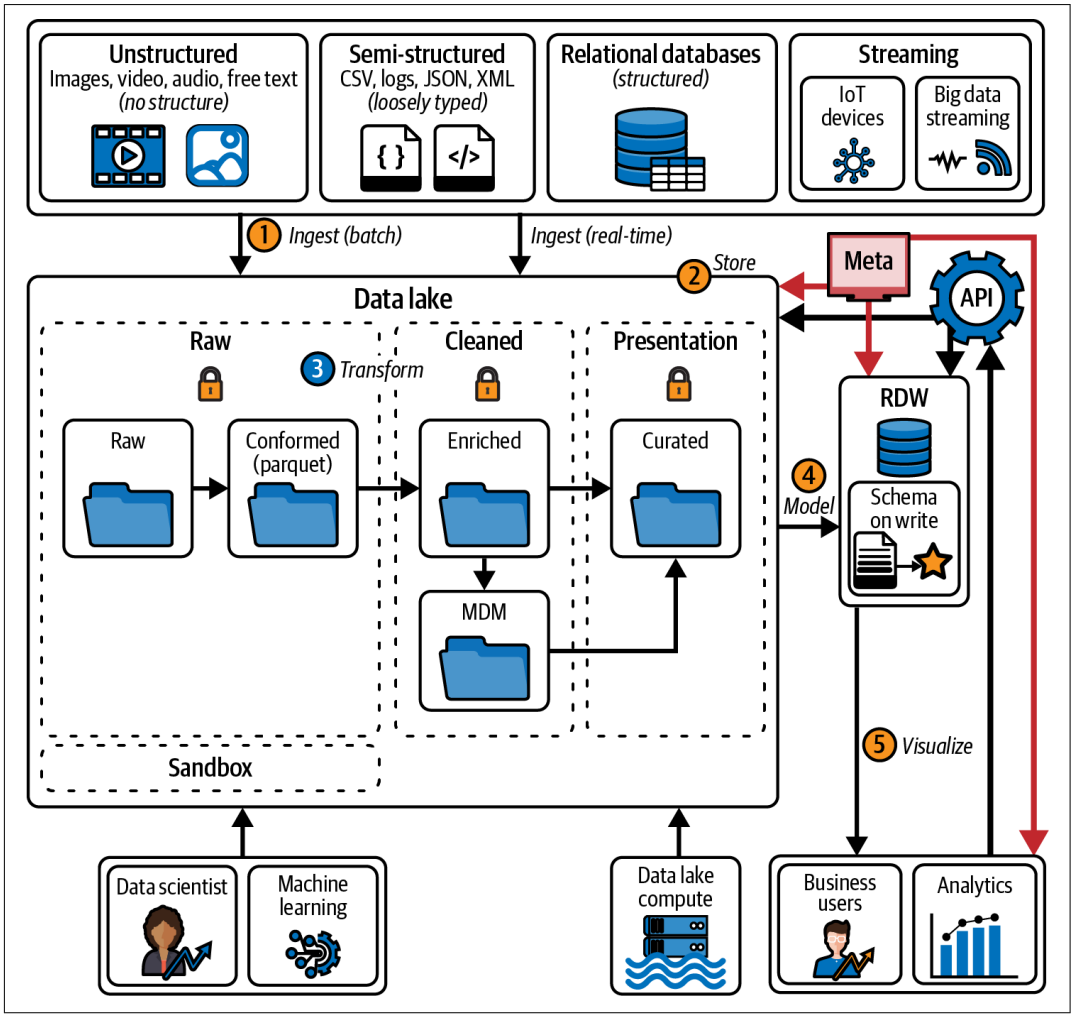

Figure 11-1. An overview of the journey data takes through a data fabric architecture

Data Access Policies

Data access policies are the key to data governance. They comprise a set of guidelines, rules, and procedures that control who has access to what information, how that information may be used, and when access can be granted or denied within an organization. They help ensure the security, privacy, and integrity of sensitive data, as well as compliance with laws and regulations, such as the UK's General Data Protection Regulation (GDPR) and the US's Health Insurance Portability and Accountability Act (HIPAA). These policies typically cover topics like data classification, user authentication and authorization, data encryption, data retention, data backup and recovery, and data disposal.

All data requests within an organization must adhere to its established data access policies and must be processed through specific mechanisms, such as APIs, drivers, or data virtualization, ensuring compliance with regulations such as HIPAA. For instance, a healthcare organization's data access policies might require that only authorized medical staff have access to patient medical records. These policies should outline procedures for verifying staff credentials and what uses of the information are acceptable. To facilitate this controlled access, the organization might implement APIs that interface with secure authentication systems. With this security in place, when a health care provider requests a patient's record, the API checks the provider's credentials against the access policies before granting access.

Metadata Catalog

Data fabrics include a *metadata catalog*: a repository that stores information about data assets, including their structure, relationships, and characteristics. It provides a centralized, organized way to manage and discover data, making it easier for users to find and understand the data they need. For example, say a user searches the metadata catalog for "customer." The results will include any files, database tables, reports, or dashboards that contain customer data. Making it possible to see what ingestion and reporting has already been done helps everyone avoid duplicate efforts.

An important part of the catalog is *data lineage*, a record of the history of any given piece of data, including where it came from, how it was transformed, and where it is stored. Data lineage is used to comply with regulations such as data privacy laws. It's also where you can trace the origin of data and the transformations it has undergone, which is an important part of understanding how it was created and how reliable it is. You wouldn't want to base important decisions on data unless you knew you could trust it! For example, if a user asks about a particular value in a report, you can look up its lineage in the metadata catalog to show how that value came about.

Master Data Management

As you learned in Chapter 6, MDM is the process of collecting, consolidating, and maintaining consistent and accurate data from various sources within a company to create a single authoritative source for master data. This data is typically nontransactional and describes the key entities of a company, such as customers, products, suppliers, and employees.

MDM helps companies make informed decisions and avoid data-related problems such as duplicate records, inconsistencies, and incorrect information.

Data Virtualization

You also encountered data virtualization in Chapter 6. To refresh your memory, data virtualization is a software architecture that allows applications and end users to access data from multiple sources as if it were stored in a single location. The virtualized data is stored in a logical layer that acts as an intermediary between the end users and the data sources. That virtual layer is a single point of access that abstracts the complexity of the underlying data sources, making it easy for end users to access and combine data from multiple sources in real time without needing to copy or physically integrate it.

Using data virtualization does not mean you need to connect *all* data sources. In most cases, it's used only for a small number of situations, with most data still being centralized. Some in the industry see data virtualization as a definitive feature of a data fabric, arguing that an architecture is not a data fabric without virtualization; I disagree. In my definition of data fabric, virtualization is optional.

Real-Time Processing

As you saw in Chapter 9, *real-time processing* refers to processing data and producing immediate results as soon as that data becomes available, without any noticeable delay. This allows you to make decisions based on up-to-date information (for example, traffic information while driving).

APIs

Instead of relying on connection strings, APIs provide data in a standardized way from a variety of sources—such as a data lake or the RDW—without sharing the particulars of where the data is located. If the data is moved, you only need to update the API's internal code; none of the applications calling the API are affected. This makes it easier to take advantage of new technologies.

APIs can also incorporate multiple types of security measures. They can also provide flexibility with security filtering, providing fine-grained control over which users can access which data.

Services

The data fabric can be built in "blocks," which make it easier to reuse code. For example, there may be others within your company who do not have a use for the full data fabric, but who do need the code you created to clean the data or to ingest it. You can make that code generic and encapsulate it in a service that anyone can use.

Products

An entire data fabric can be bundled as a product and sold. This would be particularly attractive if made specifically for an industry, such as healthcare.

Why Transition from an MDW to a Data Fabric Architecture?

There are several reasons organizations decide to transition from an MDW to a data fabric. While MDWs have traditionally been the foundation for many businesses' data infrastructures, they can be rigid and are sometimes hard to scale as data types continue to evolve rapidly. In contrast, a data fabric is designed with scalability in mind. Its inherent flexibility allows it to adapt readily to various data types and sources, making it resilient to current data demands and, to some extent, future-proof.

In addition, the sheer variety of data sources can be overwhelming. A data fabric can weave varied sources together seamlessly, offering a singular, unified view of the data that makes it much easier and more efficient to handle multifaceted data streams.

In a swiftly changing business environment, the ability to process data in real time and derive immediate insights is crucial. A data fabric meets this need by supporting real-time data processing, granting businesses instant access to the latest data.

Furthermore, as concerns about data breaches increase and the emphasis on adhering to data protection regulations intensifies, data security and governance become even more critical. A data fabric addresses these issues with advanced access policies, providing a more secure data environment. It also enhances data governance by restricting access to properly authorized users, protecting sensitive information from potential threats. And for multinational companies that operate across different jurisdictions, each with distinct data protection regulations, a data fabric's advanced governance capabilities help can maintain compliance no matter where the data is.

Lastly, certain industries—like stock trading and ecommerce—are characterized by rapidly shifting market conditions. In these volatile worlds, staying updated in real time is not just an advantage but a necessity. This immediacy is where a data fabric proves invaluable; its support for real-time data processing keeps such businesses agile, informed, and ready to adapt at a moment's notice.

Potential Drawbacks

While a data fabric architecture has numerous advantages, it isn't devoid of challenges. Transitioning from an MDW to a data fabric can be resource intensive, with initial hiccups related to cost, training, and integration. Also, not all businesses need the advanced capabilities of a data fabric—for small businesses with limited data sources and straightforward processing needs, an MDW might suffice.

Moreover, the inherent complexity of a data fabric can make troubleshooting more challenging. It's crucial to ensure that your organization has the required expertise in-house or accessible through partners before making the shift.

While data fabric offers a cutting-edge solution to modern data management challenges, it's essential for every business to evaluate its unique needs, potential return on investment, and long-term strategic goals before making the transition.

Summary

In this chapter, we delved into the concept of data fabric architecture, an advanced evolution of the MDW designed to enhance data accessibility, security, discoverability, and availability. I defined data fabric and contrasted my definition with industry perspectives, highlighting differences such as the role of data virtualization. The chapter outlined eight key technologies that transition an MDW into data fabric territory, without making a strict demarcation between the two.

I also explained the core components of the data fabric architecture, including data access policies, metadata catalog, master data management, real-time processing, APIs, and optional elements like data virtualization. The chapter emphasizes data fabric's adaptability, especially in handling large data volumes and real-time processing demands.

We looked at reasons for transitioning from an MDW to a data fabric, as well as potential drawbacks such as complexity and resource-intensive transitions. In short, every organization should carefully evaluate its unique needs and capabilities before embracing this cutting-edge solution.